

Computational Intelligence I: Neural Networks

Chapter 7: Support Vector Machines



Separation margin

⌘ To define SVMs, assume you have the training dataset $D = \{\mathbf{x}_i, d_i\}$, $i=1, \dots, N$, where the $d_i = \pm 1$ correspond to two class labels, and all $\mathbf{x}_i$ are linearly separable patterns.

⊙ Then we can find a set of weight vectors $\mathbf{w}$ and biases b such that:

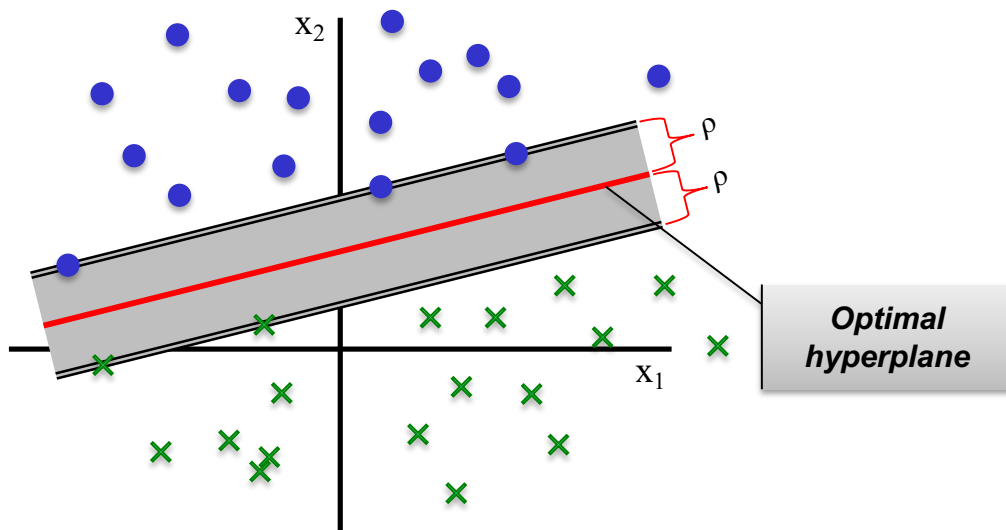
$$\begin{cases} \mathbf{w}^T \mathbf{x}_i + b \geq 0 & \forall i \in \{1, \dots, N\} : d_i = +1 \\ \mathbf{w}^T \mathbf{x}_i + b < 0 & \forall i \in \{1, \dots, N\} : d_i = -1 \end{cases}$$

⊙ For each $\mathbf{w}$ and b , we can define the **Margin of separation** ρ , to be the distance between the decision plane and the nearest point in D .

⊙ The principal aim of SVM is simply to find the **Optimal hyperplane** defined as $(\mathbf{w}_0, b_0)$ and by $g(\mathbf{x})=0$, which maximises ρ .

$$g(\mathbf{x}) \equiv \mathbf{w}_0^T \mathbf{x} + b_0$$

⊙ The discriminant function $g(\mathbf{x})$ gives a measure of the distance of $\mathbf{x}$ from the optimal hyperplane.

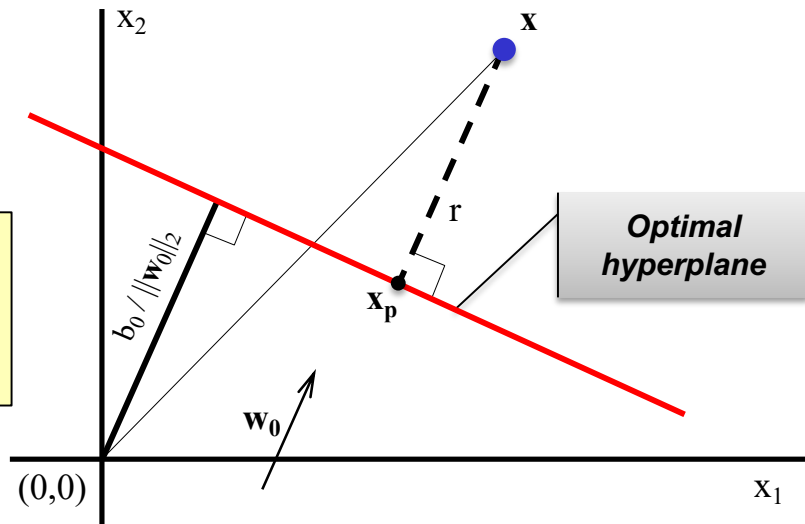


⌘ Continued...

- © For convenience, we express some point $\mathbf{x}$, in terms of its orthogonal projection $\mathbf{x}_p$ onto the Optimal hyperplane $(\mathbf{w}_0, b_0)$ and the distance r between $\mathbf{x}$ and $\mathbf{x}_p$:

$$\mathbf{x} = \mathbf{x}_p + r \frac{\mathbf{w}_0}{\|\mathbf{w}_0\|_2}$$

This is because $\mathbf{w}_0$ is perpendicular to the optimal plane defined by $\mathbf{w}_0^T \mathbf{x} + b = 0$.



- © We can see that r is positive or negative depending on which side of the hyperplane $\mathbf{x}$ lies.
- © Inserting the new expression for $\mathbf{x}$ into the discriminant $g(\mathbf{x})$, we have:

$$\begin{aligned} g(\mathbf{x}) &= \mathbf{w}_0^T \mathbf{x} + b_0 = \mathbf{w}_0^T \left(\mathbf{x}_p + r \frac{\mathbf{w}_0}{\|\mathbf{w}_0\|_2} \right) + b_0 = \\ &= \underbrace{\mathbf{w}_0^T \mathbf{x}_p + b_0}_{g(\mathbf{x}_p)=0} + r \frac{\mathbf{w}_0^T \mathbf{w}_0}{\|\mathbf{w}_0\|_2} = r \frac{\|\mathbf{w}_0\|_2^2}{\|\mathbf{w}_0\|_2} = r \|\mathbf{w}_0\|_2 \end{aligned}$$

- © Thus, we now have an expression of the distance r between $\mathbf{x}$ and the optimal hyperplane in terms of $g(\mathbf{x})$ and $\mathbf{w}_0$:

$$r = \frac{g(\mathbf{x})}{\|\mathbf{w}_0\|_2}$$

Support vectors

⌘ Our aim is (still!) to find the optimal hyperplane ($\mathbf{w}_0, b_0$) given the training set D , assuming (for now) linear separability.

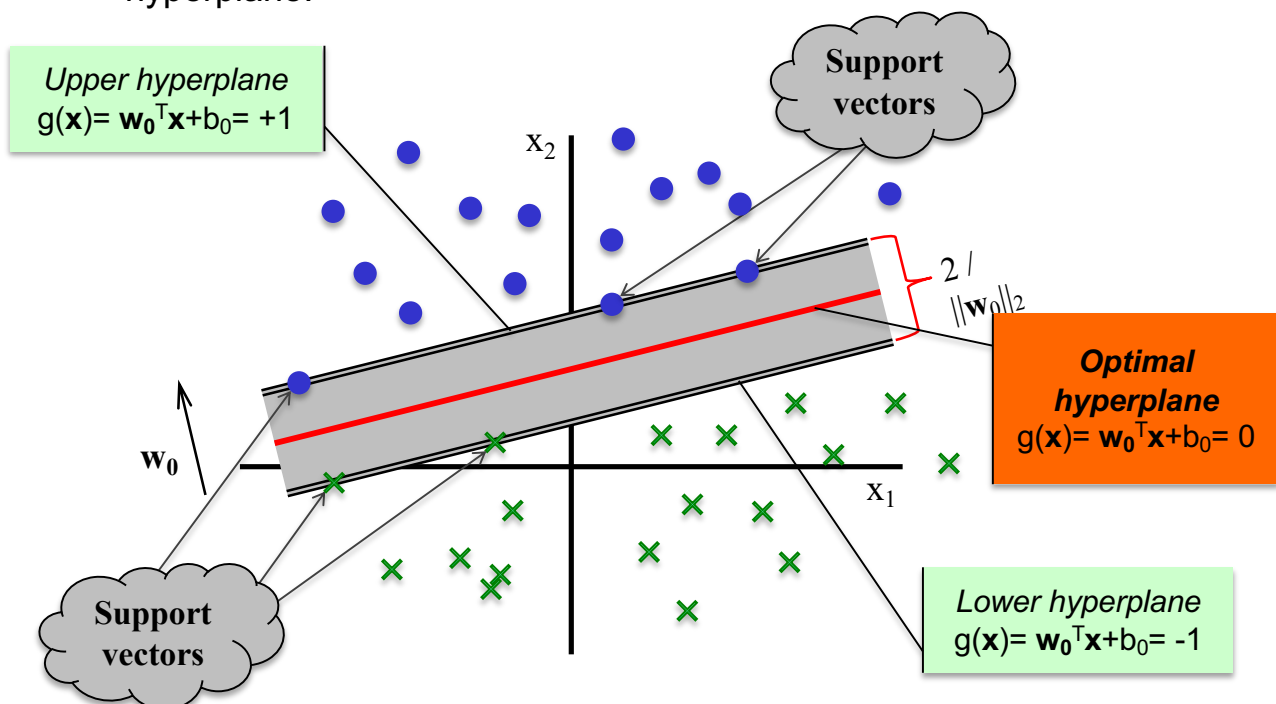
⊙ This is equivalent to finding the margin of separation such that:

$$\begin{cases} \mathbf{w}_0^T \mathbf{x}_i + b_0 \geq +1, & \forall i \in \{1, \dots, N\} : d_i = +1 \\ \mathbf{w}_0^T \mathbf{x}_i + b_0 \leq -1, & \forall i \in \{1, \dots, N\} : d_i = -1 \end{cases}$$

or simply: $\left\{ \left(\mathbf{w}_0^T \mathbf{x}_i + b_0 \right) d_i \geq +1, \quad \forall i \in \{1, \dots, N\} \right\}$

⊙ Note, that in the above, $\mathbf{w}_0$ & b_0 can be re-scaled to have ± 1 in the RHS, since they do not affect $g(\mathbf{x})=0$.

⊙ Those points $\mathbf{x}_i$ in D , that satisfy the above as an equality, are called the **Support vectors** of the dataset. These points are the most difficult to classify and are very important for the location of the optimal hyperplane.



⌘ Continued...

- © It can be seen from the figure, that the distance from the optimal hyperplane to a support vector $\mathbf{x}_s$, is:

$$r_s = \frac{g(\mathbf{x}_s)}{\|\mathbf{w}_0\|_2} = \begin{cases} +\frac{1}{\|\mathbf{w}_0\|_2} & \text{if } d_s = +1 \\ -\frac{1}{\|\mathbf{w}_0\|_2} & \text{if } d_s = -1 \end{cases}$$

- © Using now, the previously defined margin of separation ρ , we see that the goal is to solve the following maximisation problem:



$$\max_{\rho=\rho(D)} \left\{ 2\rho = 2r_s = \frac{2}{\|\mathbf{w}_0\|_2} \right\}$$

- © This shows that maximising the margin of separation or the separation between the two boundaries, is **equivalent to minimising** the Euclidean norm $\|\mathbf{w}_0\|_2$ of the weights.
- © The key point here is that the optimal hyperplane is **unique** since the optimum weight vector $\mathbf{w}_0$ provides the wider possible separation zone between the two classes.

SVM training

- ⌘ So far we have defined the SVM model, which needs to be "identified" or "selected" using D in order to be of any use.
- ⌘ Till this moment, we have ignored the fact that D may not be linearly separable (we will address this later), and assume that the previous model has a solution.
- ⌘ The optimisation problem is solved efficiently through the procedure described in the following steps.
 - © We express the Primal problem which has a convex quadratic objective function and linear constraints as:

$$\text{Primal problem} \quad \left\{ \begin{array}{l} \min_{(\mathbf{w}, b) \in \mathbb{R}^{d+1}} \frac{1}{2} \mathbf{w}^T \mathbf{w} \\ \text{s.t.} \quad d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad \forall i \in \{1, \dots, N\} \\ \text{or constraint: } h(\mathbf{x}_i) \leq 0 \end{array} \right.$$

- © Following calculus and optimisation theory, we use the Lagrange multipliers λ_i and formulate the Lagrangean function:

$$L(b, \mathbf{w}, \lambda) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + \sum_{i=1}^N \lambda_i \underbrace{\left[1 - d_i(\mathbf{w}^T \mathbf{x}_i + b) \right]}_{h(\mathbf{x}_i)}$$

- © Differentiating $L(\cdot)$, we get:

$$\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_{i=1}^N \lambda_i d_i = 0$$

$$\frac{\partial L}{\partial \mathbf{w}} = 0 \Rightarrow \sum_{i=1}^N \lambda_i d_i \mathbf{x}_i = \mathbf{w}$$

⌘ Continued...

© Using the latter two equalities, the Lagrangean becomes:

$$\begin{aligned}
 L(b, \mathbf{w}, \lambda) &= \frac{1}{2} \mathbf{w}^T \mathbf{w} + \sum_{i=1}^N \lambda_i - \underbrace{\mathbf{w}^T \sum_{i=1}^N \lambda_i d_i \mathbf{x}_i}_{\mathbf{w}} + \underbrace{b \sum_{i=1}^N \lambda_i d_i}_{\frac{\partial L}{\partial b} = 0} = -\frac{1}{2} \mathbf{w}^T \mathbf{w} + \sum_{i=1}^N \lambda_i = \\
 &= -\frac{1}{2} \left(\sum_{i=1}^N \lambda_i d_i \mathbf{x}_i \right)^T \left(\sum_{i=1}^N \lambda_i d_i \mathbf{x}_i \right) + \sum_{i=1}^N \lambda_i = \boxed{\sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j d_i d_j \mathbf{x}_i^T \mathbf{x}_j}
 \end{aligned}$$

© The above expression of the Lagrangean does **not** need the weight parameters $\mathbf{w}$ and b anymore! 

© This means we only need to optimise the Dual problem with respect to the multipliers λ_i :

$$\text{Dual problem} \left\{ \begin{array}{l} \max_{\lambda \in \mathbb{R}^N} \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j d_i d_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{s.t.} \quad \sum_{i=1}^N \lambda_i d_i = 0 \\ \lambda_i \geq 0 \end{array} \right.$$

Nonseparable patterns

⌘ But in practice no datasets are ideally linearly separable. This means that some data points in D are bound to be misclassified with a linear hyperplane.

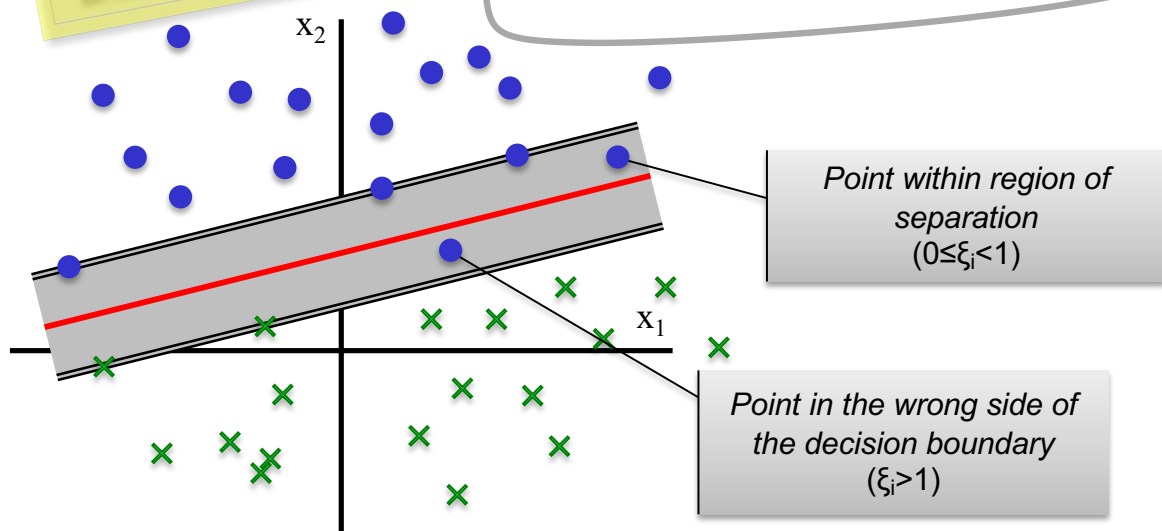
© This is relatively easily remedied with **slack variables** $\xi_i \geq 0$ which measure deviation of each i^{th} point from the ideal situation. Then, we can relax the previous constraints as:

$$d_i(\mathbf{w}^T \mathbf{x} + b) \geq 1 - \xi_i$$

© We now modify the original problem to incorporate the above. We must consider not only that we want to maximise the margin as before, but also that we need to keep all slacks ξ_i as small as possible to minimise the misclassification errors. The modified Primal problem now is:

$$\begin{aligned} \min_{(\mathbf{w}, b) \in \mathbb{R}^{d+1}} \quad & \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & \left. \begin{aligned} d_i(\mathbf{w}^T \mathbf{x} + b) &\geq 1 - \xi_i \\ \xi_i &\geq 0 \end{aligned} \right\} \forall i \in \{1, \dots, N\} \end{aligned}$$

$C \geq 0$ is a user defined parameter, which controls the **regularisation**. This is the trade-off between complexity and nonseparable patterns.

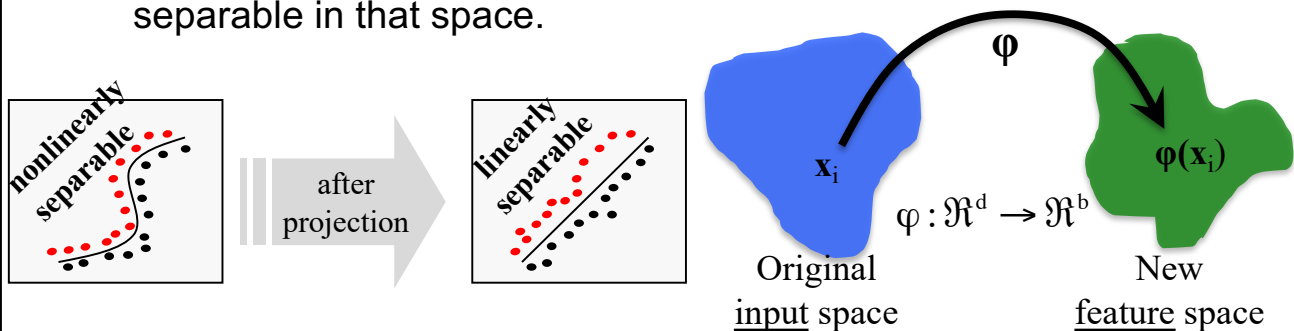


Kernels

⌘ But we are still far from the ideal case, which handles problems which require nonlinear decision boundaries.

⌘ To modify our SVM model to handle those cases, we use Cover's theorem, we described before for RBF NNs:

- ⊙ This allows us to project each patterns $\mathbf{x}_i$ to a new feature space of higher dimensionality, in order to make the patterns linearly separable in that space.



- ⊙ So, instead of changing our SVM model to a nonlinear decision boundary, we still use a linear decision hyperplane but in the new feature space.

- ⊙ This is easily achieved by defining a hyperplane in the feature space:

$$\mathbf{w}^T \varphi(\mathbf{x}) + b = \sum_{i=1}^b w_i \varphi_i(\mathbf{x}) = 0$$

- ⊙ We use our previous definition for the vector function φ :

$$\varphi(\mathbf{x}) = [\varphi_1(\mathbf{x}), \varphi_2(\mathbf{x}), \dots, \varphi_b(\mathbf{x})]^T : \mathcal{R}^d \rightarrow \mathcal{R}^b$$

- ⊙ We now use our previous SVM analysis, but we simply replace all instances of " $\mathbf{x}$ " with " $\varphi(\mathbf{x})$ " !

⌘ Continued...

- © For example, the maximising function of the previously presented Dual problem can be re-expressed as:

$$\max \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j d_i d_j \underbrace{\varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j)}_{K(\mathbf{x}_i, \mathbf{x}_j)}$$

- © The function $K(\cdot, \cdot)$ is called the **Inner product kernel**, and it is used intensely in a large class of Machine Learning algorithms, often referred to as **Kernel machines**.

$$K(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j) = \sum_{k=1}^b \varphi_k(\mathbf{x}_i) \varphi_k(\mathbf{x}_j) \quad : \quad \mathfrak{R}^d \times \mathfrak{R}^d \rightarrow \mathfrak{R}$$

- © To apply an SVM you never have to consider the explicit form of φ . Only the kernel function $K(\cdot, \cdot)$ needs to be defined.
- © There are many different types of kernels to fit the needs of the problem at hand:

Inner product kernel	Expression $K(\mathbf{x}, \mathbf{y})$	Comments
Polynomial	$(\mathbf{x}^T \mathbf{y} + 1)^p$	p is a user defined parameter
RBF	$\exp\left(-\ \mathbf{x} - \mathbf{y}\ _2^2 / (2\sigma^2)\right)$	σ is the user defined width parameter
Hyperbolic tangent	$\tanh(\alpha \mathbf{x}^T \mathbf{y} + \beta)$	α & β are user defined parameters

<END of Chapter 7>